

Laporan Mastering ROS for Robotics Programming (Chapter 1-8)

Pendahuluan

ROS (Robot Operating System) adalah framework fleksibel untuk membangun aplikasi robotika. Buku *Mastering ROS for Robotics Programming* membahas konsep, praktik, dan simulasi yang relevan untuk pengembangan robot menggunakan ROS. Laporan ini merangkum dan menganalisis isi dari Chapter 1 hingga Chapter 8, dengan menyertakan tangkapan layar dan langkah-langkah implementasi utama.

Chapter 1: Introduction to ROS

Analisis: Bab ini memperkenalkan dasar-dasar ROS, termasuk konsep seperti ROS Master, nodes, parameter server, komunikasi antar-node melalui topic, service, dan action.

Langkah-langkah:

1. Instalasi ROS:

- Instalasi dilakukan pada sistem operasi Ubuntu dengan mengikuti dokumentasi resmi ROS untuk versi yang sesuai (misalnya, ROS Noetic).

2. Mengenal Struktur ROS:

- Node: Komponen program yang berfungsi untuk melakukan tugas spesifik.
- ROS Master: Pusat komunikasi yang mengelola semua node.
- Topic: Saluran komunikasi antar-node untuk bertukar data secara asinkron.
- Service dan Action: Metode komunikasi sinkron untuk permintaan dan tanggapan.

3. Menjalankan ROS Core:

`roscore`

- roscore adalah server inti yang harus berjalan sebelum menggunakan ROS.

4. Menjalankan Node Dasar:

- Gunakan perintah `roslaunch` atau `roslaunch` untuk menjalankan node.
-

Chapter 2: Getting Started with ROS Programming

Analisis: Bab ini mengajarkan cara membuat package ROS, membangun node, dan menggunakan komunikasi melalui topic, service, serta actionlib.

Langkah-langkah:

1. Membuat ROS Package:

- o Gunakan perintah berikut untuk membuat package baru:

```
catkin_create_pkg my_package rospy std_msgs
```

- o Package ini akan memiliki dependensi rospy dan std_msgs.

2. Menulis Node Publisher:

- o Contoh implementasi node publisher dalam Python:
- o `import rospy`
- o `from std_msgs.msg import String`
- o `pub = rospy.Publisher('chatter', String, queue_size=10)`
- o `rospy.init_node('talker', anonymous=True)`
- o `rate = rospy.Rate(10)`
- o `while not rospy.is_shutdown():`
- o `pub.publish("Hello ROS")`

```
rate.sleep()
```

3. Menulis Node Subscriber:

- o Contoh node subscriber:
- o `import rospy`
- o `from std_msgs.msg import String`
- o `def callback(data):`
- o `rospy.loginfo(rospy.get_caller_id() + " I heard %s", data.data)`
- o `rospy.init_node('listener', anonymous=True)`
- o `rospy.Subscriber('chatter', String, callback)`

```
rospy.spin()
```

Chapter 3: Working with ROS for 3D Modeling

Analisis: Bab ini membahas pembuatan model robot 3D menggunakan URDF dan Xacro. Fokusnya adalah pada definisi kinematika, properti fisik, dan visualisasi model robot.

Langkah-langkah:

1. Membuat Model Robot dengan URDF:

- Buat file .urdf yang mendeskripsikan link dan joint robot.
- Contoh definisi link:
- `<link name="base_link">`
- `<visual>`
- `<geometry>`
- `<box size="1 1 1"/>`
- `</geometry>`
- `</visual>`

`</link>`

2. Menampilkan Model di RViz:

- Gunakan perintah berikut:

```
roslaunch urdf_tutorial display.launch model:=robot.urdf
```

3. Menggunakan Xacro untuk Efisiensi:

- Contoh file .xacro untuk modularitas:
- `<xacro:macro name="box_macro" params="size">`
- `<geometry>`
- `<box size="{size}"/>`
- `</geometry>`

`</xacro:macro>`

Chapter 4: Simulating Robots Using ROS and Gazebo

Analisis: Bab ini membahas simulasi robot menggunakan Gazebo, termasuk simulasi sensor dan pengendalian gerak.

Langkah-langkah:

1. Menginstal Gazebo dan ROS Plugin:

- Pastikan plugin Gazebo untuk ROS telah diinstal.

2. Memulai Simulasi Kosong:

```
roslaunch gazebo_ros empty_world.launch
```

3. Memuat Model Robot ke Gazebo:

- Gunakan perintah berikut untuk memuat model URDF:

```
roslaunch my_robot_description robot_spawn.launch
```

4. Mengontrol Robot:

- Gunakan node pengontrol seperti `teleop_twist_keyboard` untuk menggerakkan robot.

Chapter 5: Simulating Robots Using CoppeliaSim and Webots

Analisis: Bab ini mengeksplorasi simulator alternatif seperti CoppeliaSim dan Webots untuk simulasi robot.

Langkah-langkah:

1. Instalasi Simulator:

- Ikuti dokumentasi resmi untuk menginstal CoppeliaSim dan Webots.

2. Mengintegrasikan ROS dengan Simulator:

- Gunakan plugin ROS atau bridge untuk komunikasi antara ROS dan simulator.

3. Menjalankan Simulasi Robot:

- Muat model robot ke dalam simulator dan hubungkan node ROS untuk mengontrolnya.
-

Chapter 6: Using the ROS MoveIt! and Navigation Stack

Analisis: Bab ini membahas perencanaan gerak dan navigasi menggunakan MoveIt! dan Navigation Stack.

Langkah-langkah:

1. Menjalankan MoveIt! Setup Assistant:

`roslaunch moveit_setup_assistant setup_assistant.launch`

2. Konfigurasi Perencanaan Jalur:

- Tambahkan model robot dan definisi joint di MoveIt!.

3. Simulasi Navigasi dengan Navigation Stack:

`roslaunch navigation_stack navigation_demo.launch`

Chapter 7: Exploring the Advanced Capabilities of ROS MoveIt!

Analisis: Bab ini memperluas kemampuan MoveIt! untuk manipulasi objek, seperti pick-and-place dan penghindaran kolisi.

Langkah-langkah:

1. Menambahkan Objek ke Lingkungan MoveIt!:

- Gunakan API untuk mendefinisikan objek di ruang kerja robot.

2. Simulasi Pick-and-Place:

- Rencanakan jalur dengan API MoveIt! untuk mengambil dan memindahkan objek.
-

Chapter 8: ROS for Aerial Robots

Analisis: Bab ini membahas penerapan ROS pada robot udara, seperti quadcopter.

Langkah-langkah:

1. Mengintegrasikan PX4 dengan ROS:

- Gunakan plugin MAVROS untuk komunikasi.

2. Simulasi dengan RotorS:

- Jalankan simulasi quadcopter dengan RotorS:

`roslaunch rotors_gazebo mav_hovering_example.launch`

3. Mengontrol Penerbangan:

- Gunakan node kontrol untuk mengatur posisi dan kecepatan drone.
-

Kesimpulan

Laporan ini menguraikan langkah-langkah penting dari Chapter 1 hingga Chapter 8 buku *Mastering ROS for Robotics Programming*. Setiap bab memberikan wawasan dan praktik untuk memahami ROS, mulai dari dasar hingga aplikasi kompleks seperti navigasi dan robot udara. Dengan simulasi sebagai alat utama, buku ini menjadi panduan yang berguna untuk menguasai ROS.